

Front-End – Mobile Exam

I have received the details of the technical examination on 11:31 AM on June 11, 2026. Downloading the PDF, I have read through the guidelines and also downloaded the resources provided by the link.

This document will be my step-by-step documentation of how I approached this exam and the actions I have taken.

After my technical interview on June 10th, I already had the insight to create a repository for the Angular and Flutter projects in preparation for my technical exams.

Frontend Repository: https://github.com/rhenoa24/Hitachi_Angular

Mobile Repository: https://github.com/rhenoa24/Hitachi_Flutter

First, I will read through my instructions completely

The goals:

1. I have 24 hours to complete the assignment. Angular and Flutter will be used to create the project based on the guidelines. I will submit my exam for evaluation using a GitHub link.
2. Approach the project with a mobile-first design in mind.
3. Part 1: User Interface
 - a. Create a Login Screen – Screen #1
 - i. Logo display should have 3 separate icons
 - ii. Validation
 1. Button should only be clickable after user has entered a value on the input
 2. Button should be disabled if there are validation errors
 3. Keyboard must be letters only (visually for Flutter, make a Directive for Angular)
 4. Error prompts must be center aligned
 - "Please enter your username" -> Prompt of input is empty
 - "Must not exceed 24 characters"
 - "Values must be alphanumeric"
 - iii. If username is valid and entered, prompt user with 6 digit PIN
 1. Pressing close must clear PIN input
 2. Keyboard must be numbers only (visually for Flutter, make a Directive for Angular)
 3. Enter button is disabled when 6 PIN is incomplete
 - iv. Once everything is valid, direct user to Screen #2
 - b. Create a Loading Screen – Screen #2
 - i. Processes the user login and show login status to the user
 - "Logging In"
 - ii. If the API endpoint rejects the user login, return to Screen #1 with user prompt
 - iii. If API endpoint accepts, update to screen #3
 - "Fetching Data"
 - c. Create a Home Screen – Screen #3
 - i. Entire page should be built using data from API

- ii. Pressing YouTube, Spotify, Facebook should open screen with info
- iii. Pressing visit button should bring the user to the social media website (**within the app**)
- iv. Pressing 4th icon should open a screen that contains another set of brands
 - 1. Should be in a carousel, with autoplay
 - 2. Brand name below should change along logo
 - 3. Pressing visit button should launch **external browser** to brand website
- v. Pressing user avatar should open logout selection
- vi. Upon logout, redirect to Screen #2
- vii. Set a 3 second delay to simulate an API call, then return to Screen #1

What I think I need:

- 3 Main Screens
 - Login Screen
 - PIN Prompt
 - Loading Screen
 - Home Screen
 - Social Media Screen
 - Browser Display from link
 - Brand Screen
 - Autoplay Carousel
 - Logout Prompt
- Angular Directives
 - Text Only Input (expect special characters, for the validation)
 - Number Only Input

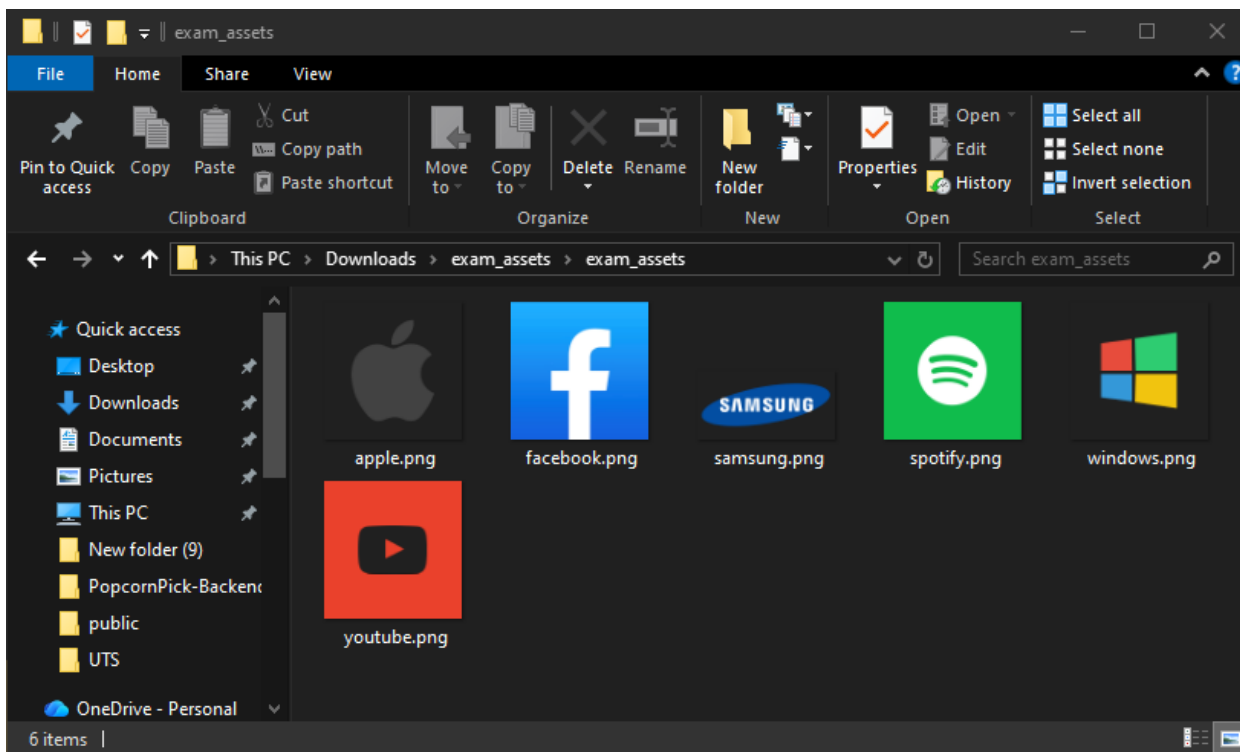
API Endpoints

Request	Response
https://indexcodex.com/api/v1/login Method: POST Header: key: CLIENT_ID value: rgbexam Body (data type: String): userName otp Notes: Any username is accepted Valid OTP values: 123456 / 123123	If success: user data If failed: error details
https://indexcodex.com/api/v1/socials Method: GET Header: key: CLIENT_ID value: rgbexam Body: N/A	If success: socials data If failed: N/A

Recreating Mockups

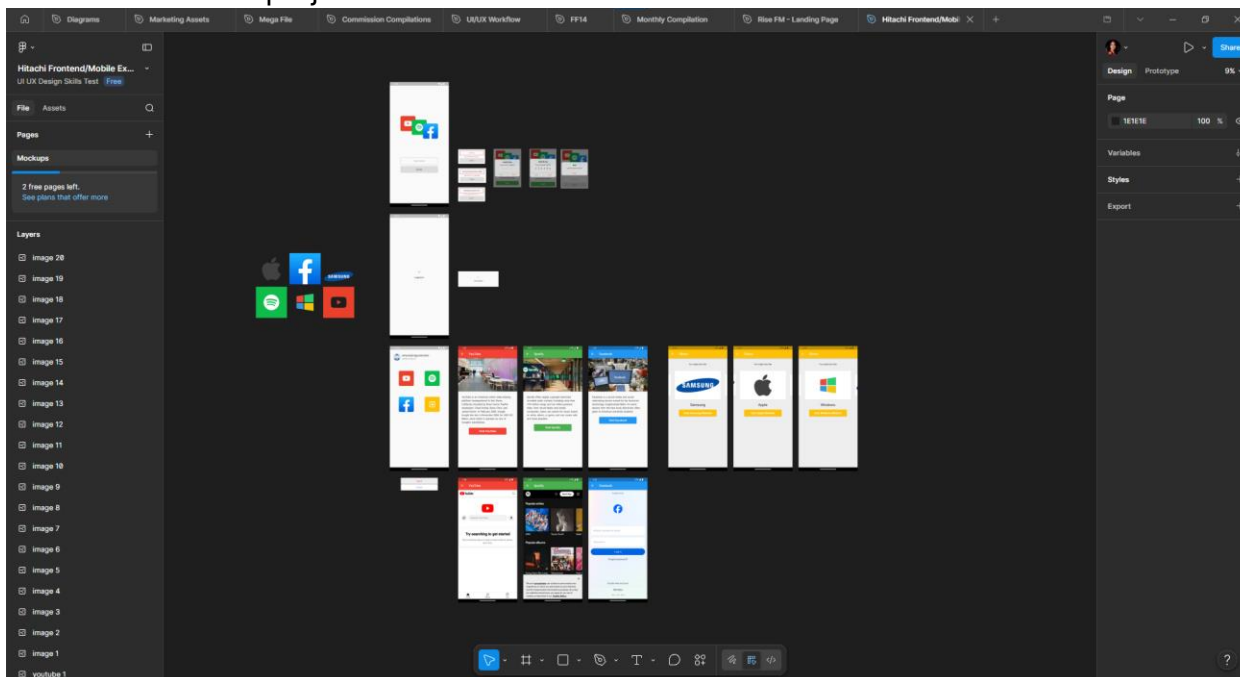
I have downloaded the exam_asset.zip and discovered the image asset within the folder.

With that, I need to recreate the mockups in the PDF so I can accurately replicate the screens



Figma Link: <https://www.figma.com/design/IXDbEVycPNQCSPC326PCh/Hitachi-Frontend-Mobile-Exam?node-id=0-1&t=PTlaKzz2wR4l1vss-1>

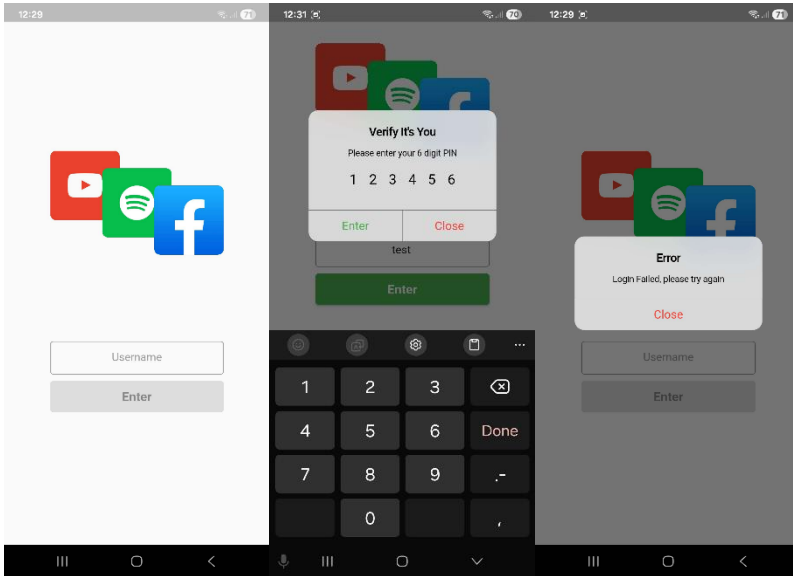
I have laid out and grouped the screens from the PDF, to get a grasp of the scale and have dimension references for the project.



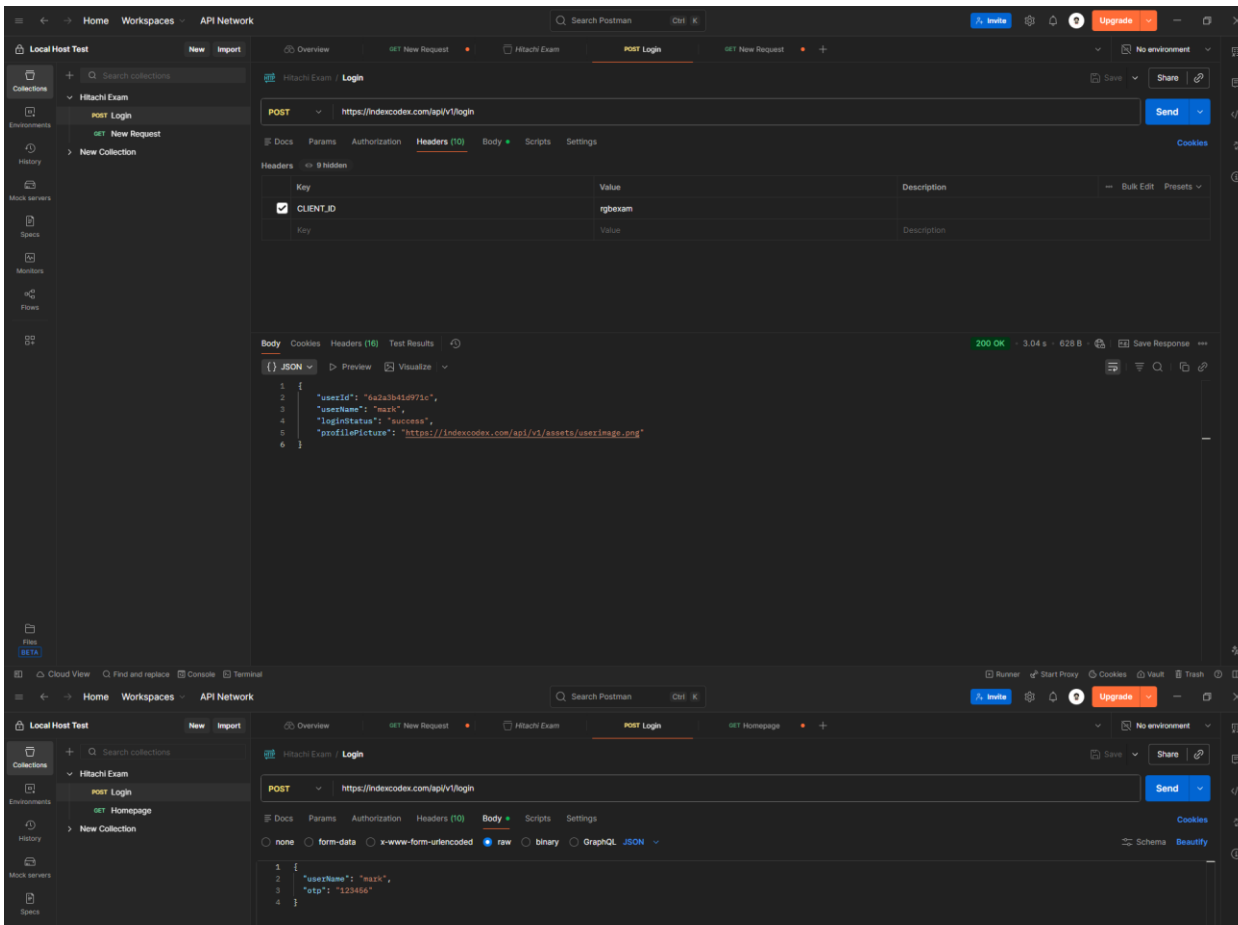
Base screen dimension: W: 970px x H: 2048px

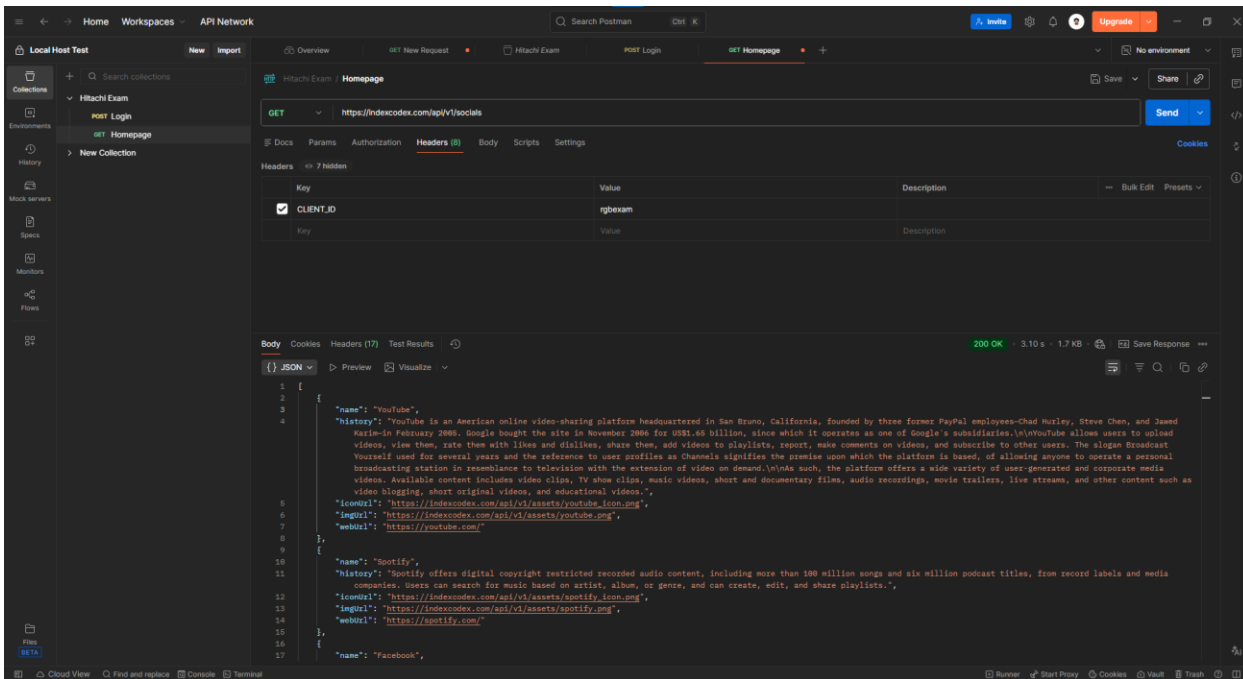
Testing API via Postman

Before I start on anything, I want to test the references given to me. I have also installed the APK provided in the PDF on my personal phone. I encountered a problem with the login so I figured I should test the API itself first if the provided URL works.



I have tested in Postman and confirm that the API works!





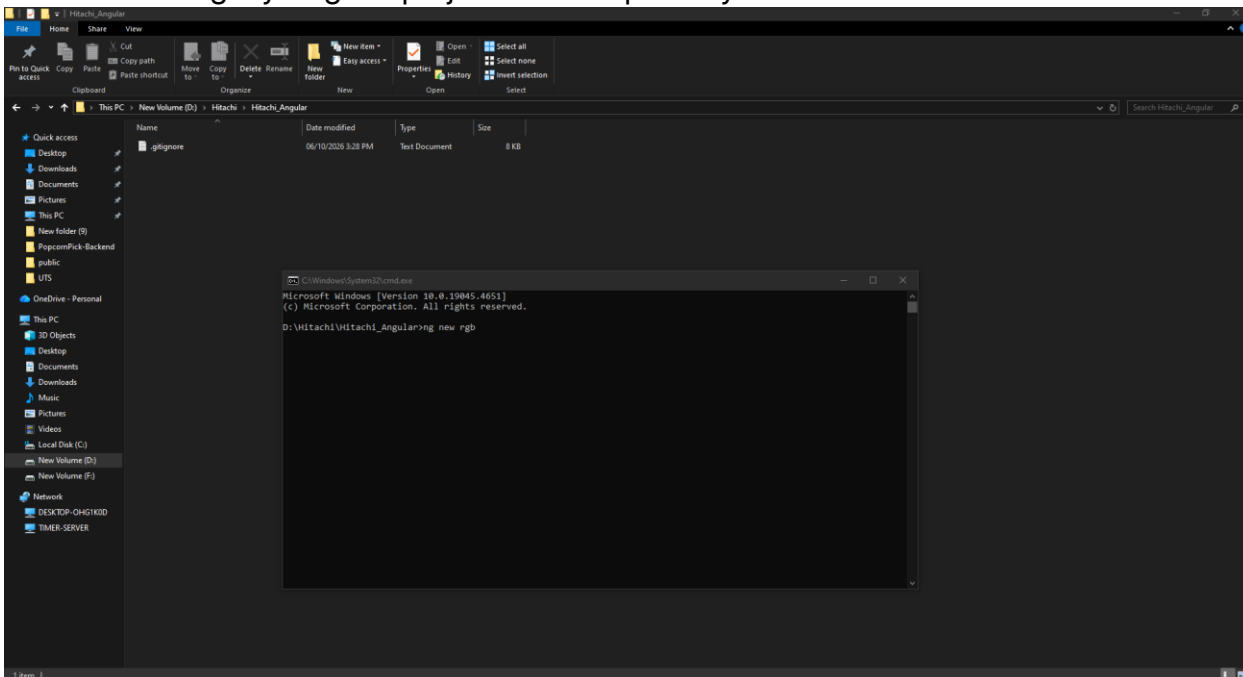
I have also noticed that the app is called “rgb” so I will take that as reference.

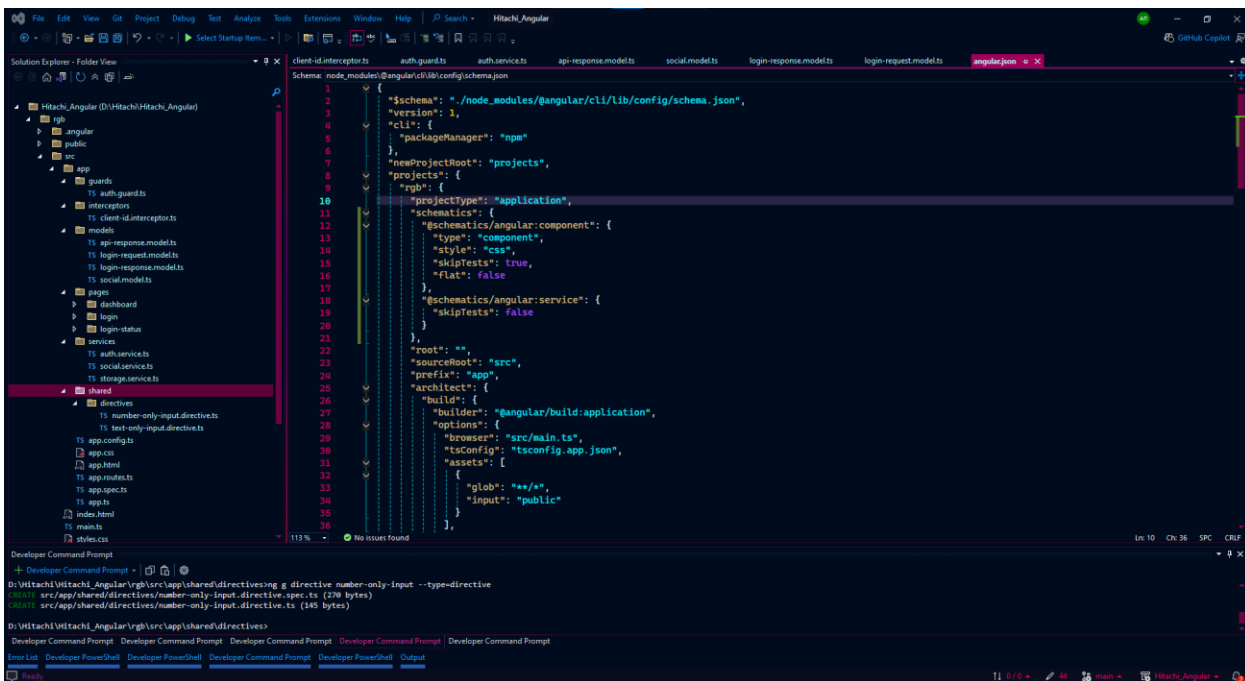
Angular Project

Since I am most experienced with Angular, I will be starting the project first with this.

Frontend Repository: https://github.com/rhenoa24/Hitachi_Angular

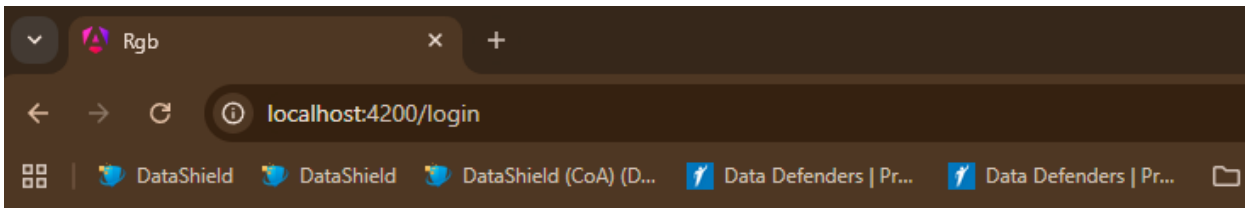
I will be creating my Angular project in this repository.





So far at 2PM after eating lunch, I have mostly committed the Frontend Skeleton I have in mind. Fixing the imports, doing dependency injections, and updating the routes.

I plan to create a low fidelity HTML template just to check the API working on the Angular project.



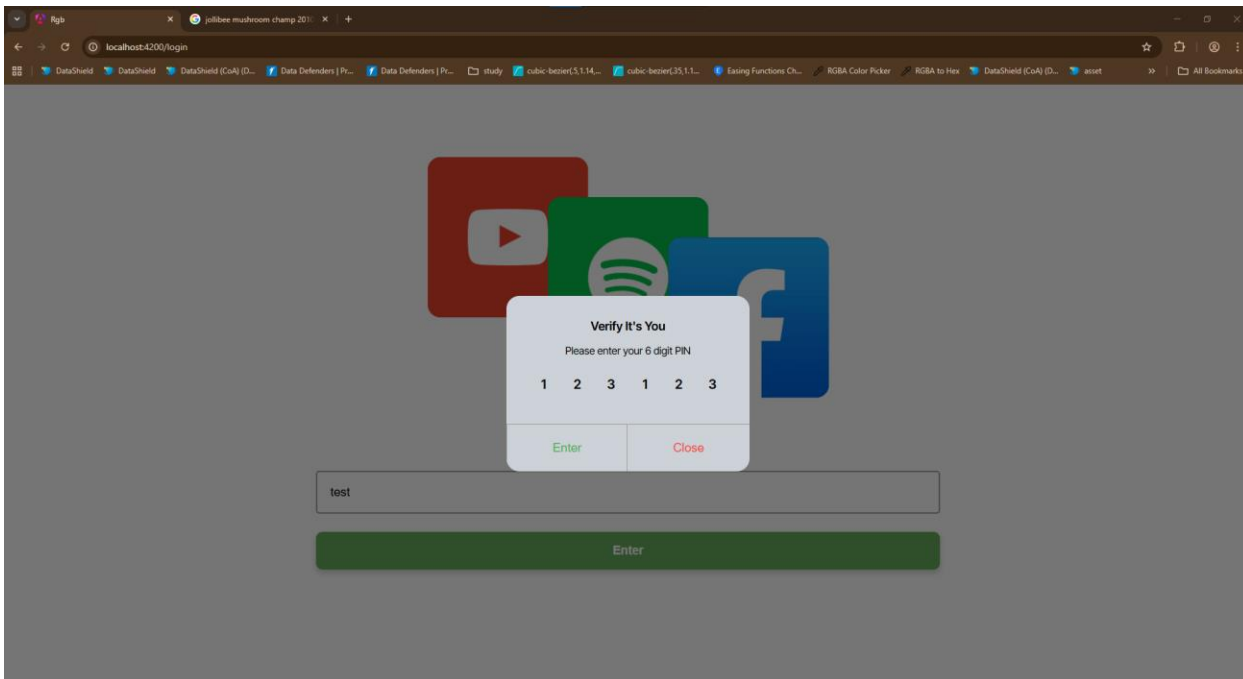
Login Page

Enter Username

Values must be alphanumeric

Login

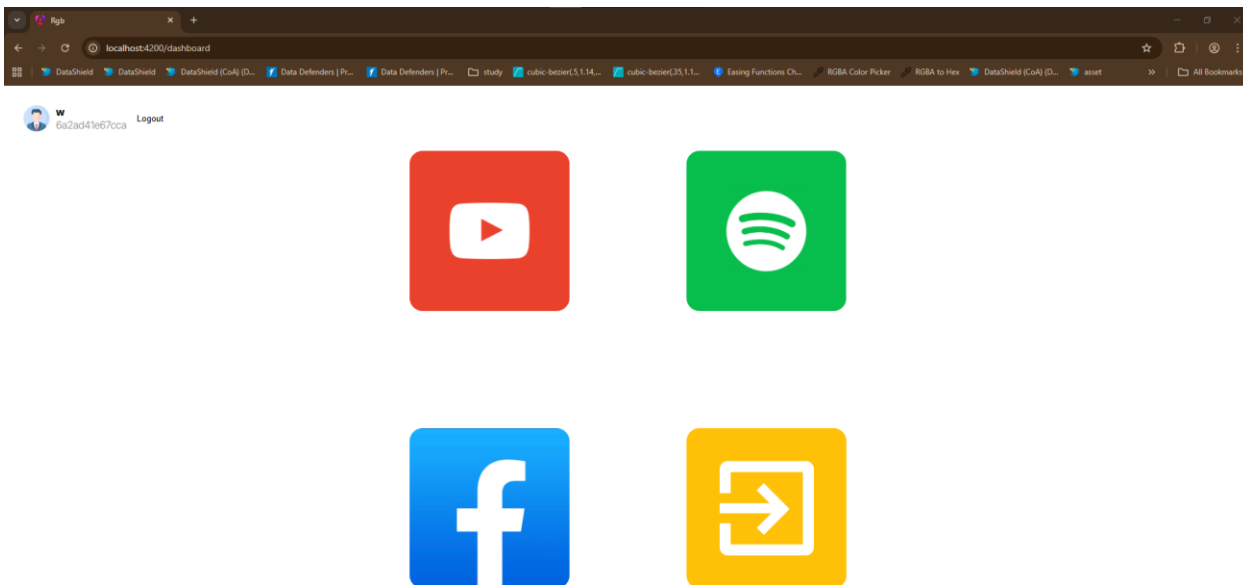
Around 6pm, I have a fully working low fidelity website. I have validated my services and API are working. I'm proceeding to recreating the mockups.

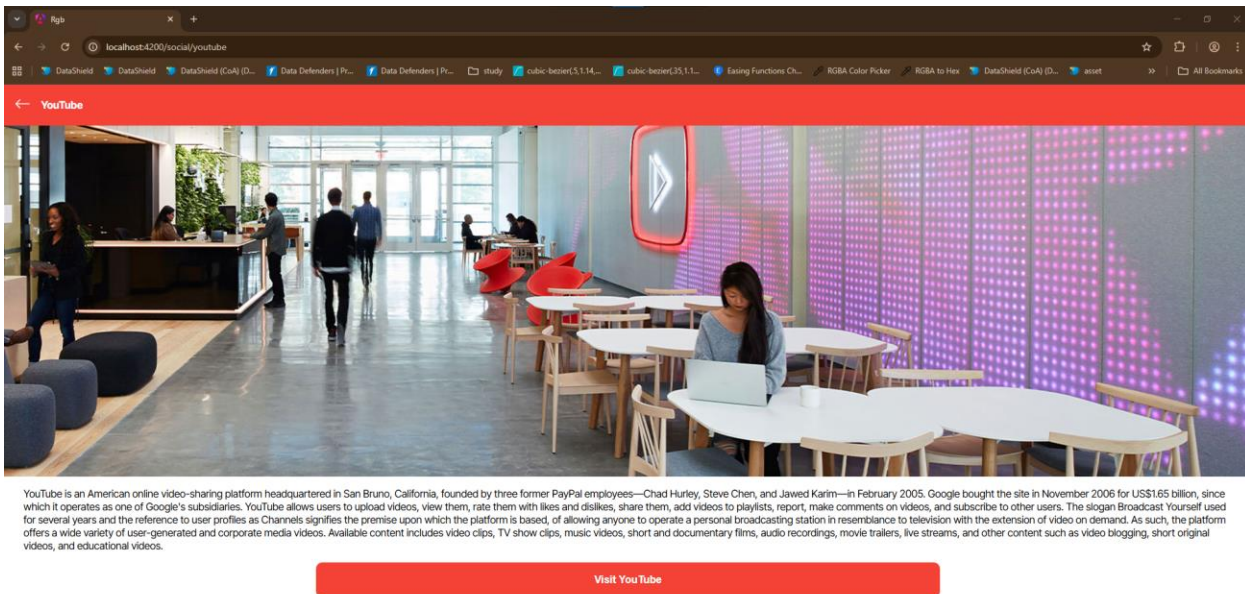


Progress as of 9PM

- Created similar Login Page
- Create OTP Input Component as it is too complex to leave the logic in the login.component
- Form Validations, Error popups
- Loader Component polished

With the login page polished and tested, I will now move to the Dashboard.



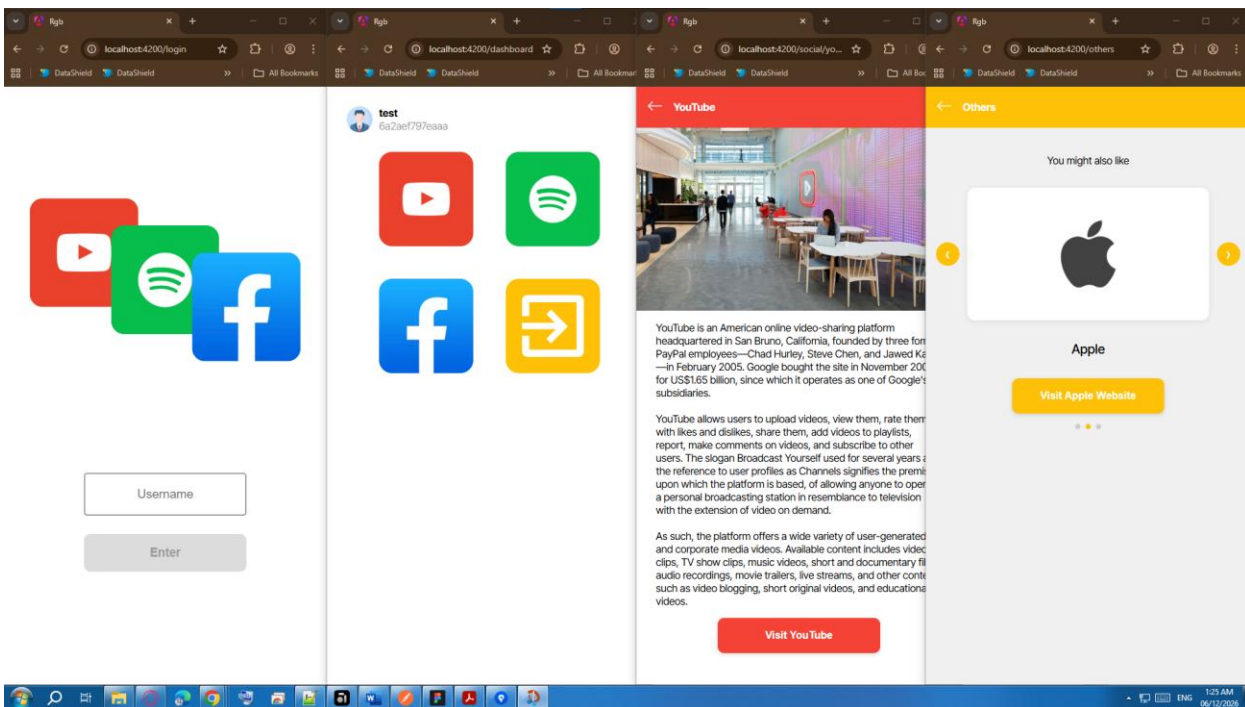


As of 11PM, I managed to finish the Socials UI. Still going to work on the Logout function similar to the guidelines before moving to the Others section.

In Angular, `<iframe>` isn't working out to be similar to the instructions. Youtube, Facebook, and Spotify are refusing to connect, so I just made it open the link in a new tab.

I kept the components in, just for posterity. It has the URL sanitization which I thought would display it.

As of 1AM, I have completed the Angular Project and added in some UX behavior as well. My plan is to convert my Angular logic to Flutter.

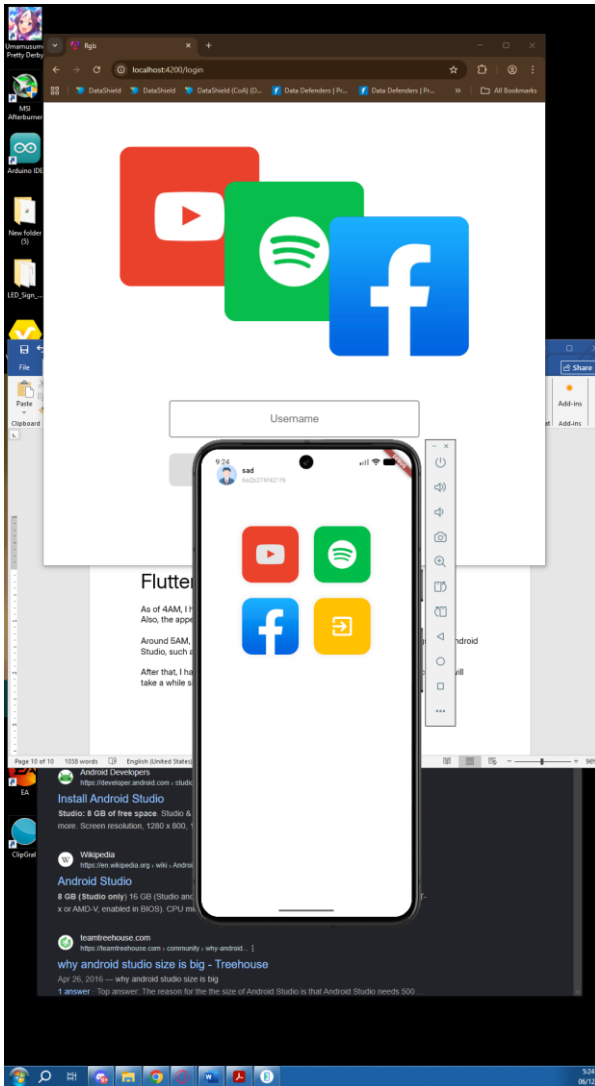


Flutter Project

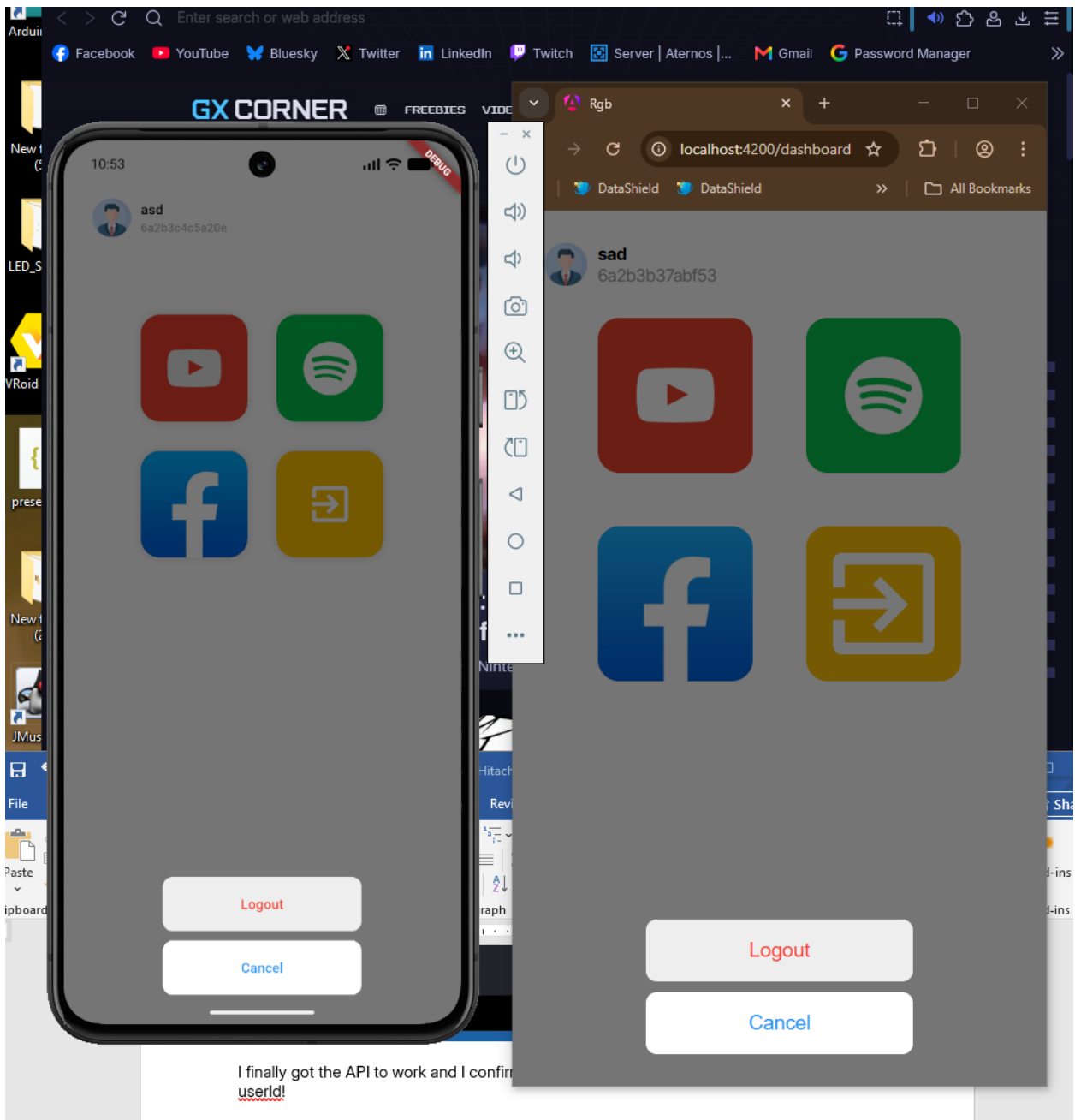
As of 4AM, I have only updated the skeleton of my Angular pages.
Also, the appearance and input validation of the Login page is fixed.

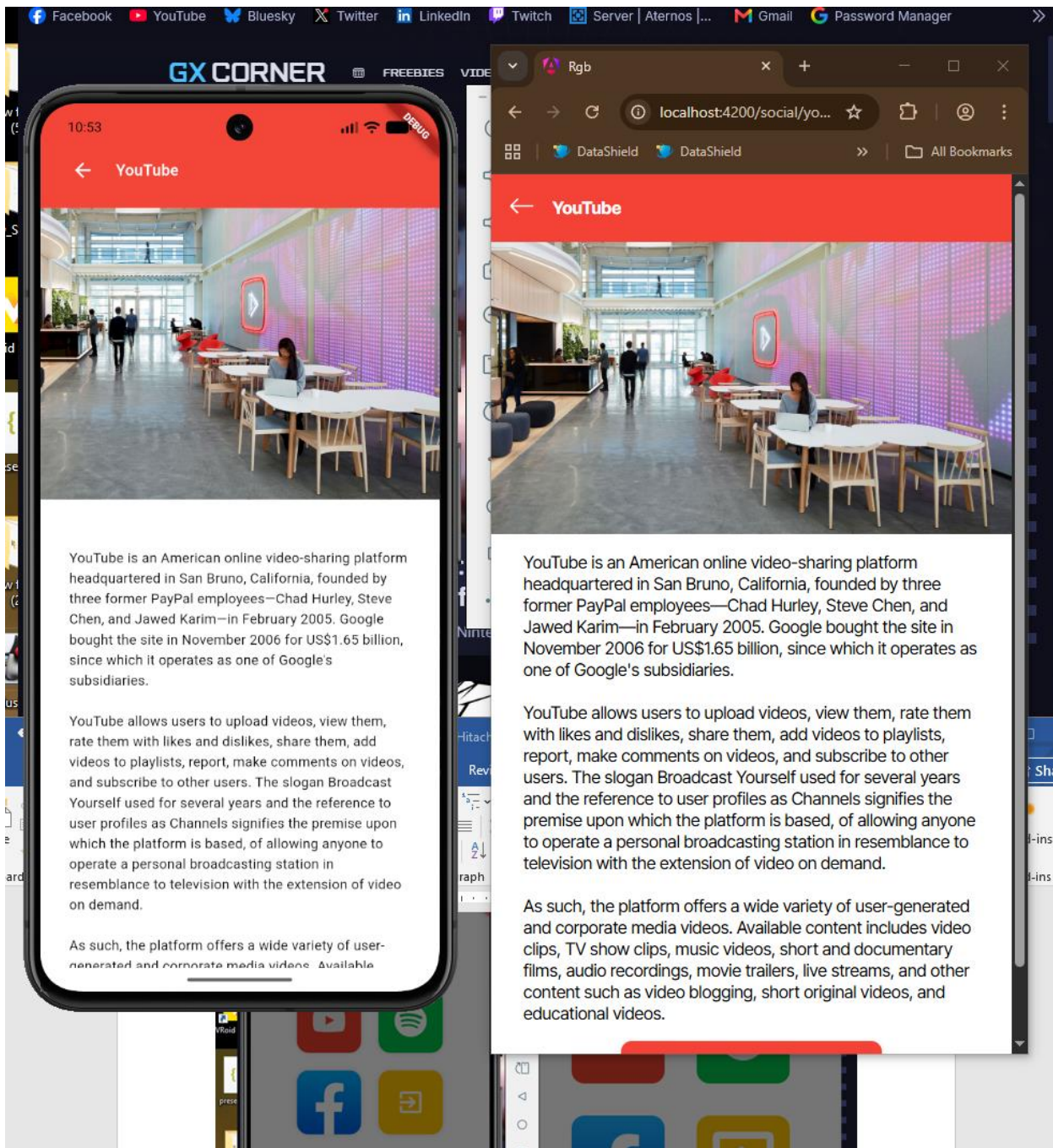
Around 5AM, I have finished downloading and installing the necessary things with Android Studio, such as the Android SDK Command-line Tools (latest)

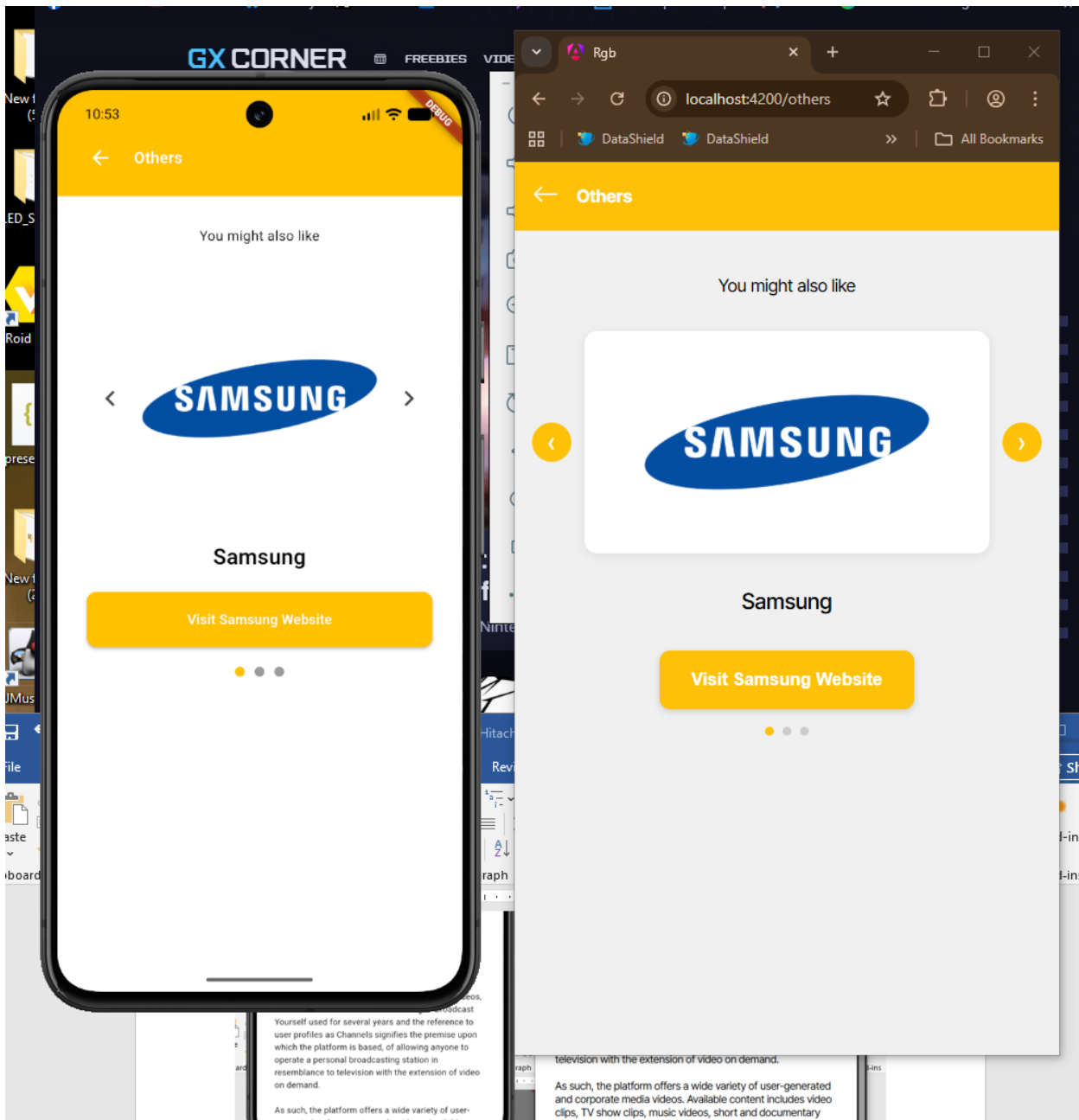
After that, I have encountered that I have to also download for a virtual device. This will take a while since it is 2.1GB.



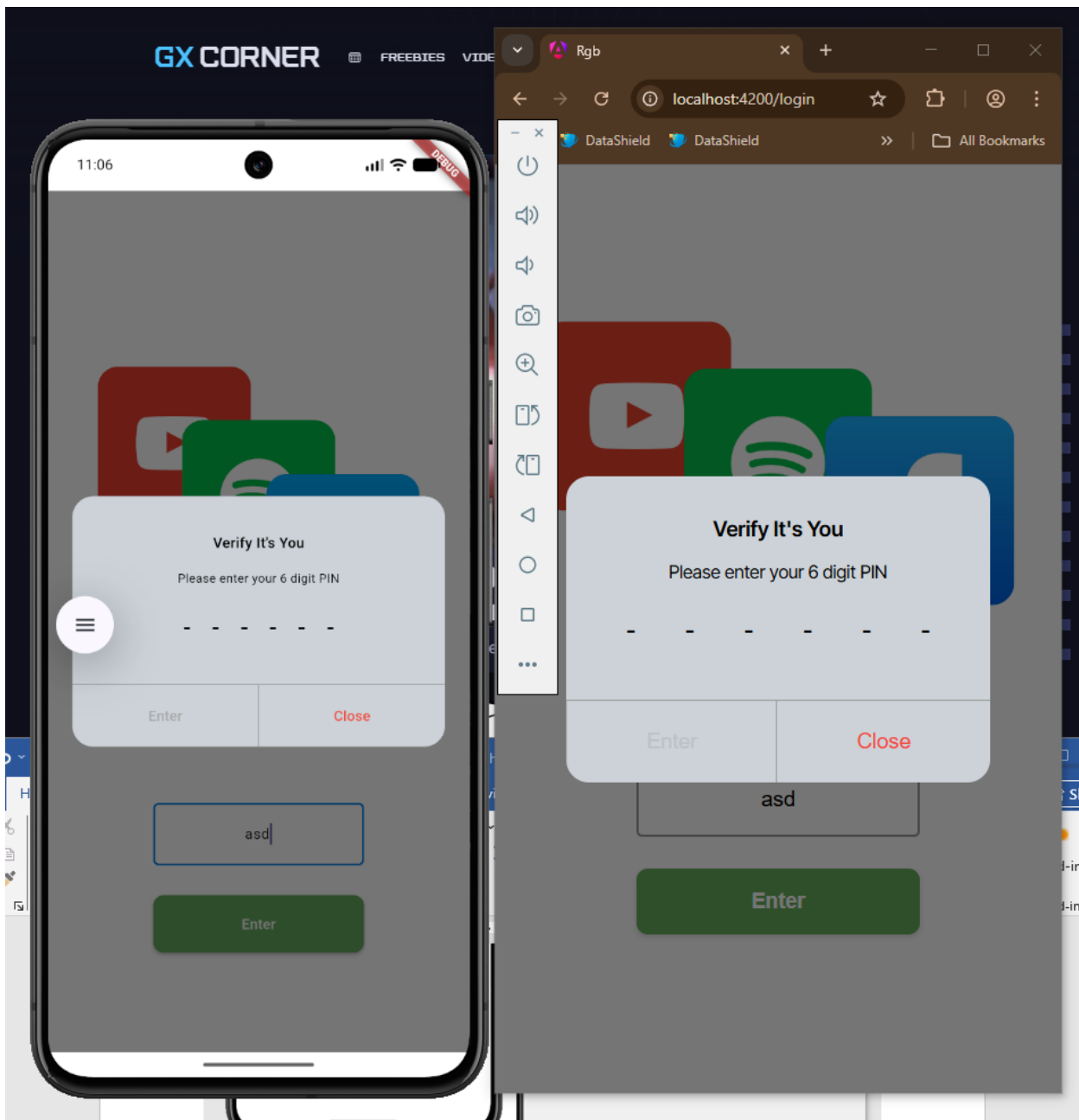
I finally got the API to work and I confirm the APIs work based on the userId!



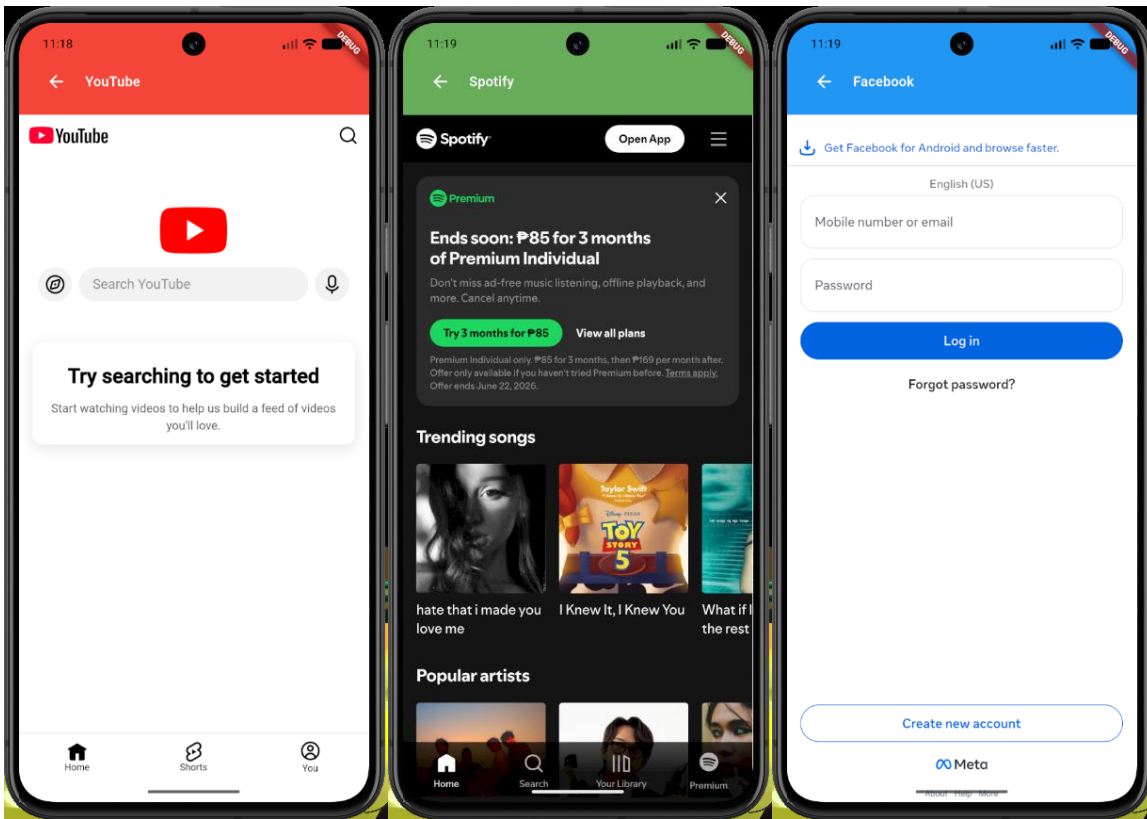




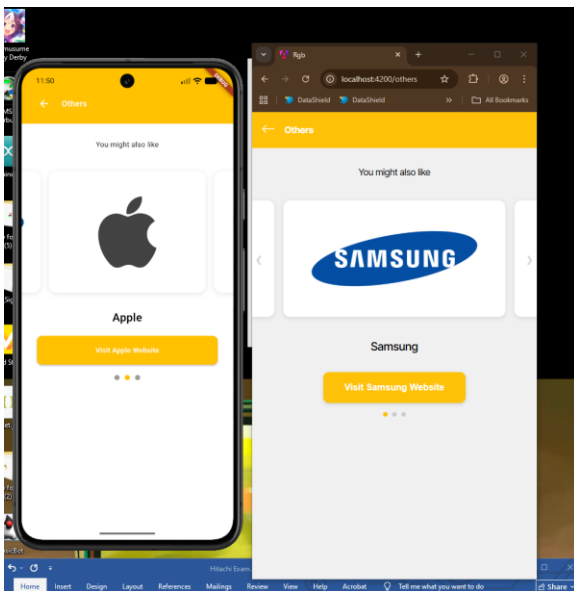
At 7AM, I was pretty much done with the Flutter App. Finally got the match the OTP popups.



OTP UI finally matches well on both Angular and Flutter.



Finally got this to work in Flutter! Since Angular / Chrome are very picky! It's nice to see it in action.

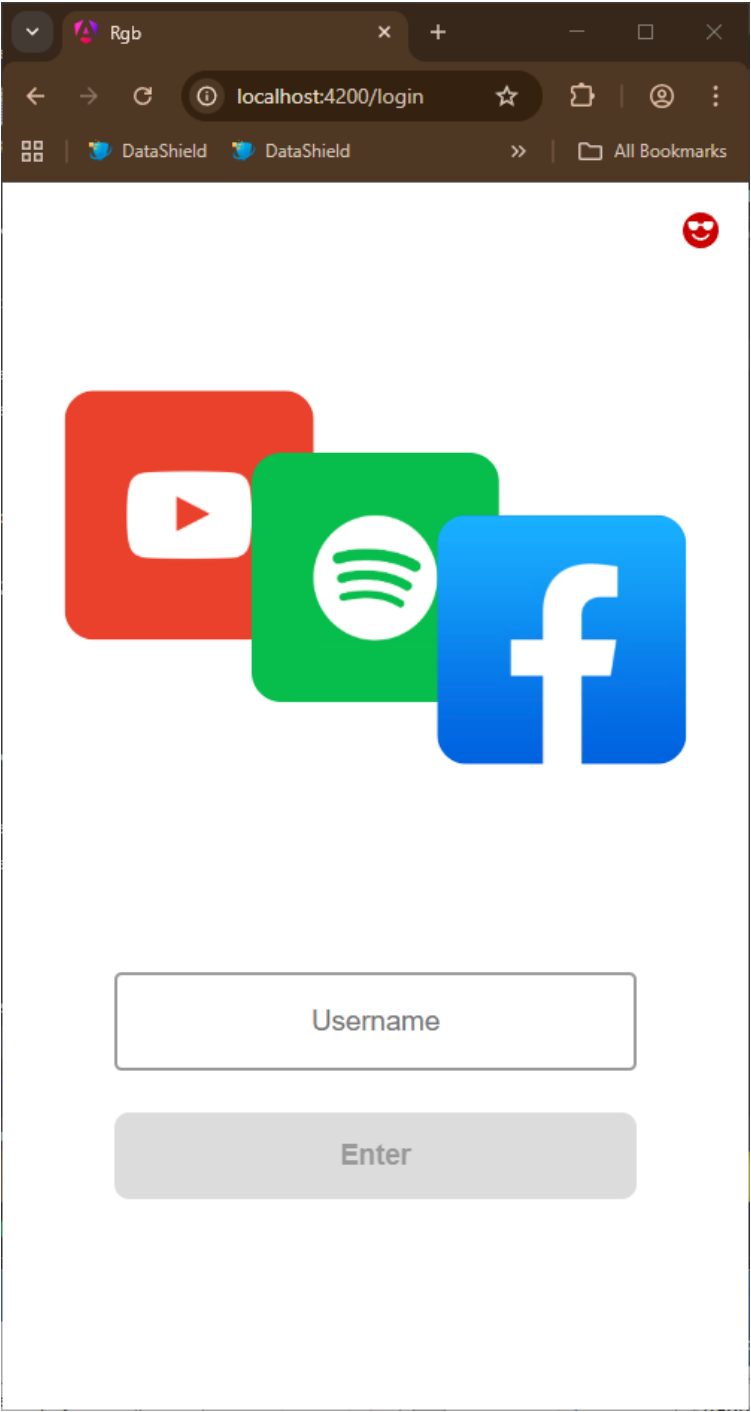


The carousels resemble the guidelines much closely. Angular doesn't have the drag functionality at the moment. I'll think about it.

By 9:15AM, I am fully done with the exam and will be going to sleep.

Aly's Easter Egg

On the top right of the Angular project, there is a hidden icon. Click it and have fun with dark theme!



Code Review – Angular

My Structure:

environment

environment.ts → for the apiUrl: '/api'

guards

auth.guard.ts → if user is not logged in, navigate back to login.

Connected to AuthService, uses isLoggedIn

interceptors

client-id.interceptor.ts → to automatically attach CLIENT_ID: 'rgbexam' header to every outgoing HTTP request

models

login-request.model.ts → request body for POST method of login
(username, otp)

login-response.model.ts → what POST method returns / response model
(userId, username, loginStatus, profilePicture)

social.model.ts → response model for GET of socials data
since it's a GET, no need for request body
(name, history, iconUrl, imgUrl, webUrl)

pages

dashboard → displays data fetched by social.service.ts

others → contains hard-coded brands

login → uses auth.service.ts

services → inject HttpClient and import Models

auth.service.ts → login() is http.post with url and payload
setSession() stores to localStorage
logout() removes from localStorage
isLoggedIn() checks if localStorage is not null
getSession() returns save data from localStorage

social.service.ts → getSocials() returns the socials\$
socials\$ is an Observable doing the http.get
with a .pipe(shareReplay(1)) that caches the result
shareReplay to any new subscriber

theme.service.ts → personally added, stores and gets theme from localStorage

shared

components

loading, **otp-input**, **social-card**, ~~social-viewer~~, theme

css

app.config.ts, app.css, app.html, app.ts

app.routes.ts → canActivate: [authGuard] for routes only for logged in users

login.component

injected Router, **AuthService**, ChangeDetectorRef, Subscription
imported OtplnputComponent
UI states: step, isLoading, loadingText, loginFailed
Import **login-request.model**

Imported ReactiveFormsModule for form validation, created loginForm FormGroup
showError() returns bool flag if input has been interacted (dirty / touched)
getErrorMessage() returns string according to .hasError()

submitUsername()

Updates the step and runs otplnput.focusInput()
Return if reactive form input is invalid

submitOtp() → Where I trigger the login API call = POST request

Building payload using reactive form control values
Set isLoading = true and loadingText = "Logging In"

this.loginSub = **this.authService.login(payload).subscribe({})**

next: (res) → API succeeds

JSON.stringify(result) and store in localStorage

isLoading = false

Navigate to dashboard and cdr.detectChanges() for good measure

error: (err) → API fails

Reset relevant flags and clear the reactive form using resetFields()

closeOtp() → resets OTP input and returns to username step

closeError() → for button click on Error Popup when failed login, resets loginFailed = false

resetFields() → loginForm.reset()

ngOnDestroy → I clean up with loginSub?.unsubscribe()

otp-input.component

@Input() control!: FormControl<string>

Login.component uses this as <app-otp-input [control]="loginForm.controls.otp">
[control] is one-way binding

Get value() { return this.control.value ?? " " } for the HTML

onInput() would clean the input.value

and use **control.setValue(cleaned)** and **control.markAsTouched()**

no need for @Output() or EventEmitter because of Angular reactive forms

dashboard.component

injected **SocialService**, Router, **AuthService**, Subscription, ChangeDetectorRef

UI states: **socials: Social[]**

Is Loading, loadingText, **userId**, **username**, **profilePicture**, showLogoutUI

Import **social.model**

ngOnInit() → Where I trigger the login API call = GET request

loadingText = 'Fetching Data'

runs authService.getSession()

if session exists, store the values to UI states: **userId**, **username**, **profilePicture**

this.dashboardSub = **this.socialService.getSocials().subscribe({})**

next: (**data: Social[]**) → API succeeds

this.socials = data, isLoading = false, cdr.detectChanges()

error: (err) → API fails

isLoading = false, console.error() the error, cdr.detectChanges()

logout()

Runs authService.logout(). Fake API call delay for 3 seconds.

openSocial(social: Social)

navigates to the /social page route, carrying over context

this.router.navigate(['/social', social.name.toLowerCase()], {

state: { social } → entire Social object packed here

});

openOthers() → just navigates to /others

showLogout() → toggle for showLogoutUI

ngOnDestroy → Clean up using this.dashboardSub?.unsubscribe()

app.routes.ts → I use URL parameters for the subpages dynamically made from API

path: 'social/:name',

canActivate: [authGuard],

component: SocialCardComponent

social-card.component

injects Router and social.model

social: Social = history.state.social → gets unpacked here

openViewer() opens social.webUrl to new tab

note: I do not make any API calls here so it fails when I manually enter the URL in a new tab

it works on refresh because of Angular dev server serving the page itself

the browser made the HTTP request to load the page

app.config.ts → where provideHttpClient, withInterceptors, **client-id.interceptor** is imported

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes),

    provideHttpClient(
      withInterceptors([clientIdInterceptor])
    )
  ]
}
```

App.ts → where theme.service and theme.component is imported (personal flair)

```
private themeService = inject(ThemeService);

ngOnInit(): void {
  const theme = this.themeService.getTheme();
  document.body.classList.add(`${theme}-theme`);
}
```

theme.component

A simple component to set keys for localStorage via **themeService**

The component that applies the themes via ngOnInit()

```
document.body.classList.add(`${themeKey}-theme`);
```

theme.service.ts

A simple service with getTheme() and setTheme()

Just for localStorage.setItem and .getItem

IMPORTANT: OVERCOMING CORS POLICY

Proxy.conf.json

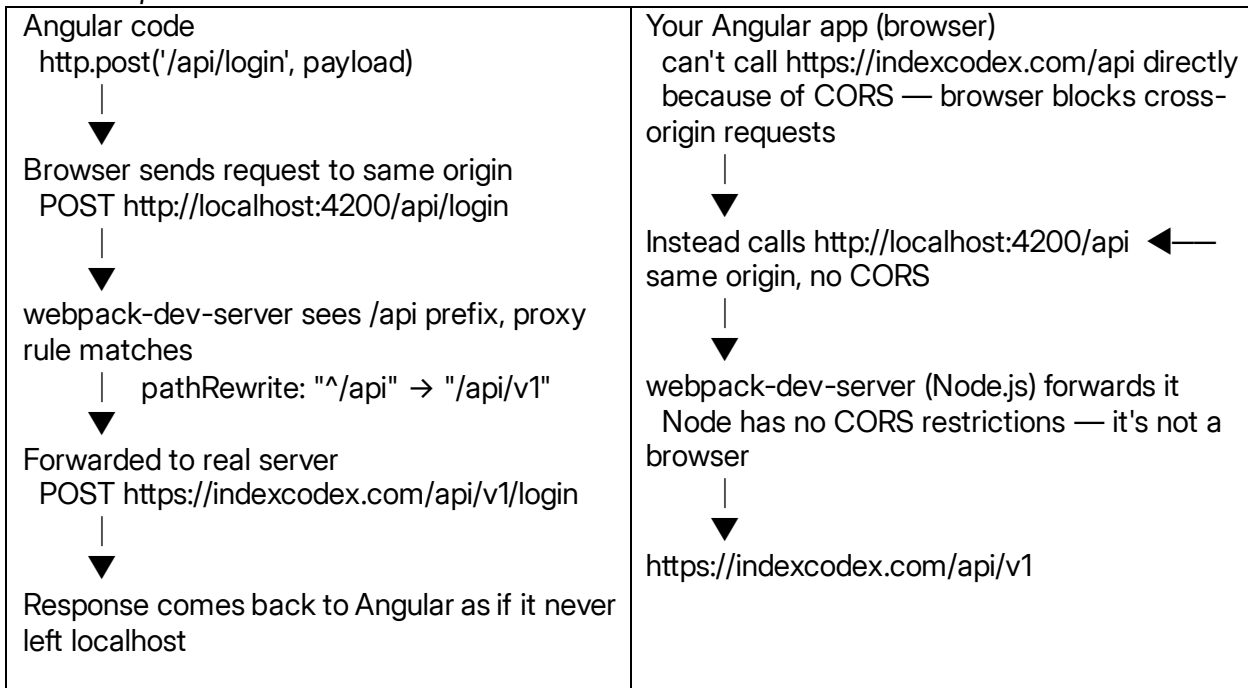
```
{
  "/api": {
    "target": "https://indexcodex.com",
    "secure": false,
    "changeOrigin": true,
    "logLevel": "debug",
    "pathRewrite": {
      "^/api": "/api/v1"
    }
  }
}
```

I had to apply this proxy by changing the following

Angular.json

```
"serve": {
  "options": {
    "proxyConfig": "proxy.conf.json"
  }
}
```

Claude Explanation:



That's the core reason the proxy pattern exists — **the browser has CORS restrictions, Node.js doesn't**. The proxy makes your API call look same-origin to the browser, then Node forwards it freely.

Code Review – Flutter

I have immediately noticed that there's no separate HTML or CSS like files. Everything is encapsulated on a .dart file, so some parts of the app have to become a separate widget as I cannot easily apply global styles like in CSS selectors.

Select Pixel 8 → flutter run

CORS policy is overcome by running the Flutter app in an Android Studio via Virtual Device Manager.

My Structure (lib):

constants

App_theme.dart, colors.dart

models

brand.dart
login_request.dart
login_response.dart
social.dart

pages

Dashboard_page.dart
Login_page.dart
Others_page.dart
Social_card_page.dart
Social_viewer_page.dart

services

auth_service.dart
social_service.dart
theme_service.dart

widgets

loading_widget.dart
others_button.dart
otp_input_widget.dart
page_header.dart
popup_modal.dart
social_card_widget.dart

main.dart → handles the routing to Social pages (where caching happens)

I am not fluent in Flutter and I have mostly used resources to convert my Angular logic in this

Key Differences VS Angular:

No separate environment → \$_baseUrl is set per service

No separate auth.guard.ts → There is Flutter equivalent but it wasn't translated along

No client-id.interceptor → Header is added in per service on the API call

No theme functionality → Just added flair in Angular proj after I finished Flutter proj